



A hybrid neural combinatorial optimization framework assisted by automated algorithm design

Liang Ma¹ · Xingxing Hao¹ · Wei Zhou¹ · Qianbao He¹ · Ruibang Zhang¹ · Li Chen¹

Received: 19 March 2024 / Accepted: 29 July 2024 / Published online: 17 August 2024
© The Author(s) 2024

Abstract

In recent years, the application of Neural Combinatorial Optimization (NCO) techniques in Combinatorial Optimization (CO) has emerged as a popular and promising research direction. Currently, there are mainly two types of NCO, namely, the Constructive Neural Combinatorial Optimization (CNCO) and the Perturbative Neural Combinatorial Optimization (PNCO). The CNCO generally trains an encoder-decoder model via supervised learning to construct solutions from scratch. It exhibits high speed in construction process, however, it lacks the ability for sustained optimization due to the one-shot mapping, which bounds its potential for application. Instead, the PNCO generally trains neural network models via deep reinforcement learning (DRL) to intelligently select appropriate human-designed heuristics to improve existing solutions. It can achieve high-quality solutions but at the cost of high computational demand. To leverage the strengths of both approaches, we propose to hybrid the CNCO and PNCO by designing a hybrid framework comprising two stages, in which the CNCO is the first stage and the PNCO is the second. Specifically, in the first stage, we utilize the attention model to generate preliminary solutions for given CO instances. In the second stage, we employ DRL to intelligently select and combine appropriate algorithmic components from improvement pool, perturbation pool, and prediction pool to continuously optimize the obtained solutions. Experimental results on synthetic and real Capacitated Vehicle Routing Problems (CVRPs) and Traveling Salesman Problems (TSPs) demonstrate the effectiveness of the proposed hybrid framework with the assistance of automated algorithm design.

Keywords Neural combinatorial optimization · Transformer · Reinforcement learning · Automated algorithm design

Introduction

The Combinatorial Optimization Problem (COP) is an important and classic problem in the field of operation research. It has a wide range of applications in various fields, such as vehicle scheduling [2], molecular prediction [3],

Sudoku solving [4], tourism route optimization [5], etc. The core of COP lies in optimizing the arrangement, grouping, ordering, or selection of discrete events to minimize or maximize an objective function subject to given constraints.

In recent years, researchers have explored the application of neural networks (NNs) and deep reinforcement learning (DRL) in solving COPs, resulting in the new taxonomy Neural Combinatorial Optimization (NCO) [1, 11, 13, 18, 20, 30, 33]. Typically, based on the workflow, NCO methods can be classified as Constructive Neural Combinatorial Optimization (CNCO) [1, 11–13, 18, 20, 39] and Perturbative Neural Combinatorial Optimization (PNCO) [20, 25, 26, 28–31, 33, 40]. The CNCO trains NNs to construct solutions from scratch. Once the parameters of NNs are learned via supervised learning or reinforcement learning (RL), it can directly output solutions of instances that have similar scale and distribution as the trained samples. This process is very quick compared with search-based approaches and requires no expert knowledge. However, it cannot guarantee the optimality of the generated solutions since they are directly

✉ Xingxing Hao
xingxing.hao@nwu.edu.cn
Liang Ma
maliang1@stumail.nwu.edu.cn
Wei Zhou
mczhouwei12@gmail.com
Qianbao He
202221477@stumail.nwu.edu.cn
Ruibang Zhang
zrb41@stumail.nwu.edu.cn
Li Chen
chenli@nwu.edu.cn

¹ Northwest University, Xi'an 710127, China

mapped from the original problems without further refinement [1]. Therefore, the CNCO is competitive in small-scale COPs but less effective in large-scale COPs compared with traditional solvers like LKH3 [6] and Google OR Tools [7]. Unlike CNCO, the PNCO follows the workflow of traditional local search. It optimizes existing solutions by perturbing or adjusting them iteratively through human-designed heuristics. The innovation lies in that it learns to automatically design or select the appropriate heuristics based on the current state instead of manual design. Therefore, it can achieve better solutions than traditional local search. However, it always trains the NN model via RL, which requires massive episodes to converge. Moreover, it heavily relies on expert knowledge and experience [30–33].

From above we can conclude that CNCO methods are time-efficient but no guarantee of solution quality due to the one-shot mapping. On the contrary, PNCO methods can achieve high solution quality with sustained optimization, but struggle with large computational burden. To leverage their strengths, i.e., CNCO can rapidly generate quasi-optimal solutions that serve as good initial states of PNCO, and PNCO can continuously optimize on the base of initial solutions, to improve the overall performance, this paper proposes to hybrid them into a unified framework assisted by automated algorithm design. We term it as Hybrid Neural Combinatorial Optimization (HNCO) framework.

The HNCO consists of two stages. In the first stage, namely, the constructive stage (HNCO-CS), the Attention Model (AM) is employed to construct solutions from scratch for COPs. The purpose of this stage is to produce quasi-optimal solutions in a quick manner, which can then be used as the initial solutions of the second stage. In the second stage, namely, the perturbative stage (HNCO-PS), we borrow the idea of automated algorithm designs that is claimed in [8] and [9], and employ the DRL to intelligently select appropriate heuristics during different stages of optimization process to iteratively improve the solutions provided by the first stage. For the second stage, we design three types of heuristic pools, namely, the improvement pool, prediction pool, and perturbation pool, each of which contains a diverse set of heuristics. It is noteworthy that in this paper, we regard the NN-based operators in the prediction pool as a form of data-driven heuristic operators. The DRL is used to automatically design algorithm that can select the appropriate heuristics from the given heuristic pools.

The merits of HNCO are two-folds: (1) Quasi-optimal solutions can be generated in a very quick manner, which can save computational cost and provide higher starting points, namely, better initial solutions, for the HNCO-PS. (2) The solutions produced by HNCO-CS can be continuously improved by the HNCO-PS, which can liberate their potential bounded by the labeled training data, thus achieve higher quality. Therefore, the HNCO framework is expected

to achieve better performance than sole CNCO and PNCO, which are indeed as our experimental results on capacitated vehicle routing problems (CVRPs) and traveling salesman problems (TSPs) will show.

Our work contributes primarily in the following three aspects:

1. We propose a hybrid framework HNCO that combines the CNCO and the PNCO in the purpose of taking their advantages to improve the overall performance of HNCO. Specifically, HNCO consists of two stages, the HNCO-CS and the HNCO-PS. In HNCO-CS, the AM is trained to construct the initial solution for COPs in a quick manner. Subsequently, HNCO-PS iteratively improves the solution output by HNCO-CS via algorithms learned by DRL.
2. For the HNCO-PS stage, three types of heuristic pools, namely, the improvement pool, prediction pool, and the perturbation pool are designed. The heuristics in the improvement pool are dedicated to fine-tuning the current solution to improve its quality. The prediction pool aims to capture characteristics of local optimal solutions of the solving COPs and predict new potential solutions. The perturbation pool is inclined to reboot the search to start a new optimizing iteration by changing the solution dramatically. These three heuristic pools can provide richer choices for the automated algorithm design process, with the expectation of enhancing the searching power of the designed algorithms.
3. The effectiveness of the proposed HNCO framework is validated on synthetic and real CVRP instances from CVRPLib and TSP instances from TSPLib. The analyses of extensive ablation experimental results confirm the necessity of the hybrid of CNCO and PNCO, demonstrate the effectiveness of the synergy of three heuristic pools and the generalizability of the automatically designed algorithms. Moreover, we visualize the selection probabilities of heuristics learned by the employed policy network, which provides interesting views and constructive proposals for future researches.

The rest of this work is organized as follows: In “[Related work](#)”, we will discuss the related work and review the existing researches on the topic. “[HNCO Framework](#)” presents the proposed HNCO. In “[Experiments](#)”, experiments are conducted to evaluate the effectiveness of the HNCO framework. Finally, “[Conclusions](#)” concludes this work.

Related work

CNCO. The Hopfield NN is an early NN model proposed for solving COPs in this field, there are still ongoing innovations

based on it [10] currently. The pointer networks [11], which is a seminal work in this field, can also be considered as a variant of the Hopfield NN. It has typical encoder-decoder structure where two separate recurrent NN (RNN) serve as the encoder and decoder, respectively. Furthermore, the encoder in [12] replaces the traditional RNN with a convolutional NN and utilizes real-world datasets for supervised learning. They are trained through supervised learning and then can generate solutions for COPs very fast in an autoregressive manner. However, its performance is bounded by the labeled data, that is, the quality of generated solutions is at best as the training data. Besides, the acquisition of high-quality training data is difficult as well, which further limits its generality. Thus, Bello et al. [13] proposed to train the Pointer Network using the REINFORCE algorithm [14], in which a Critic network was used as the baseline to mitigate the instability in policy updates. Their method exhibits better performance in TSP instances with 50 cities than supervised learning [11]. However, Pointer Network-based approaches are still inferior to traditional solvers [6, 7, 13, 15].

Later, due to its remarkable performance in natural language processing tasks, the Transformer Model (TM) has become popular in CNCO area and methods based on TM has achieved the best performance on many COPs [1, 17, 19, 20]. Compared to the Pointer Network, the multi-head attention mechanism in TM is better at capturing long-range dependency. Therefore, later works shift their focus to TM. Deudon et al. [17] first adopted the multi-head attention in the encoder and train their model via REINFORCE method. Specifically, the methodology involves utilizing a NN to construct an initial solution, followed by applying a simple 2-Opt perturbation to the initial solution. Experimental results demonstrate that this approach can effectively enhance the quality of the solutions. It further demonstrates that integrating the CNCO and PNCO models can enhance the efficiency of problem-solving. Based on Deudon's work, Kool et al. [1] proposed the Attention Model (AM) and further improved the efficiency of the decoder by using the current best model as the baseline during training instead of a trained critic network. This approach has been shown to be able to enhance learning speed and stability. As a CNCO model, its performance not only surpasses that of the pointer network model [11–13] but also closely approaches the solutions of LKH3 [6], Gurobi [15], and Concorde [6]. Since then, AM became a classic model in CNCO and subsequent research has explored various variants of the AM model [18, 21–24]. Consequently, due to the outstanding performance exhibited by the AM in CNCO, we choose it to construct initial solutions in the first stage of HNCO. However, all these methods [1, 17] take the problem instance as input and directly output solutions using trained deep NNs, with network parameters typically trained through DRL. It is challenging to guarantee the optimality of the output solution.

PNCO. Local search is a class of very efficient traditional algorithms for solving COPs. In recent years, researchers have tried to use DRL to automatically design local search algorithms to solve COPs, with the expectation of exceeding the human-designed heuristics [25, 26, 29]. Chen et al. [25] proposed a local search model, NeuRewriter, by training the strategy through DRL. NeuRewriter comprises two parts, i.e., the Region-picker and Rule-picker, where Region-picker is responsible for selecting the region for local search, while Rule-picker selects heuristics for the chosen local search region. Experiments on CVRPs show that NeuRewriter can enhance the efficiency of heuristic algorithm design. Additionally, apart from categorizing the local search model into the steps of selecting reconstruction objects and applying heuristic rules on these objects, it can be further abstracted as the stages of disruption and repair. Following this idea, Gao et al. [26] employed DRL to learn the destroy and repair heuristics for solving CVRPs and CVRP with time windows (CVRPTW), where destroy heuristics eliminates specific nodes from the current solution, and repair heuristics reintegrates the deleted nodes into the current solution based on the trained strategy. The model is trained using the Proximal Policy Optimization (PPO) algorithm [27].

Actually, the above technologies fall into the scope of the automated algorithm design [25, 26], which is also a hot topic in current researches [8, 9, 28–30]. Specifically, algorithm design can be modeled as a COP in which the decision variables are basic algorithm components and the solution is their optimal combinations that constitute an algorithm to solve given COPs [30]. Based on this, Qu et al. [30] proposed the concept of general COP (GCOP). Considering the properties of GCOP, RL can typically effectively guide heuristics in the decision-making process when solving COPs [8, 31]. For instances, [8] uses deep Q networks and PPO to select heuristic and evolution heuristics for solving CVRPTW, which marks a critical step for RL in automated algorithm design. Yi et al. [31] developed an effective meta-heuristic algorithm to address CVRPTW by utilizing PPO to analyze instance-specific and search-specific features. In addition, the L2I [33], a recently proposed effective learning-to-improve method, also belongs to the category of automatic algorithm design. It categorizes local search heuristics into improvement heuristics and perturbation heuristics. Then, the DRL is employed to train the policy network for selecting appropriate improvement heuristics in every iterative step. When the improvement stagnates, the perturbation heuristics are used to reboot the search. L2I is a successful framework that surpasses professional COP solvers like LKH3 [6] and Google OR-tools [7], in terms of efficiency and solution quality on 20-, 50-, and 100-CVRPs. Therefore, we adopt L2I as the second stage of HNCO, with a newly designed heuristic pool, namely, the prediction pool.

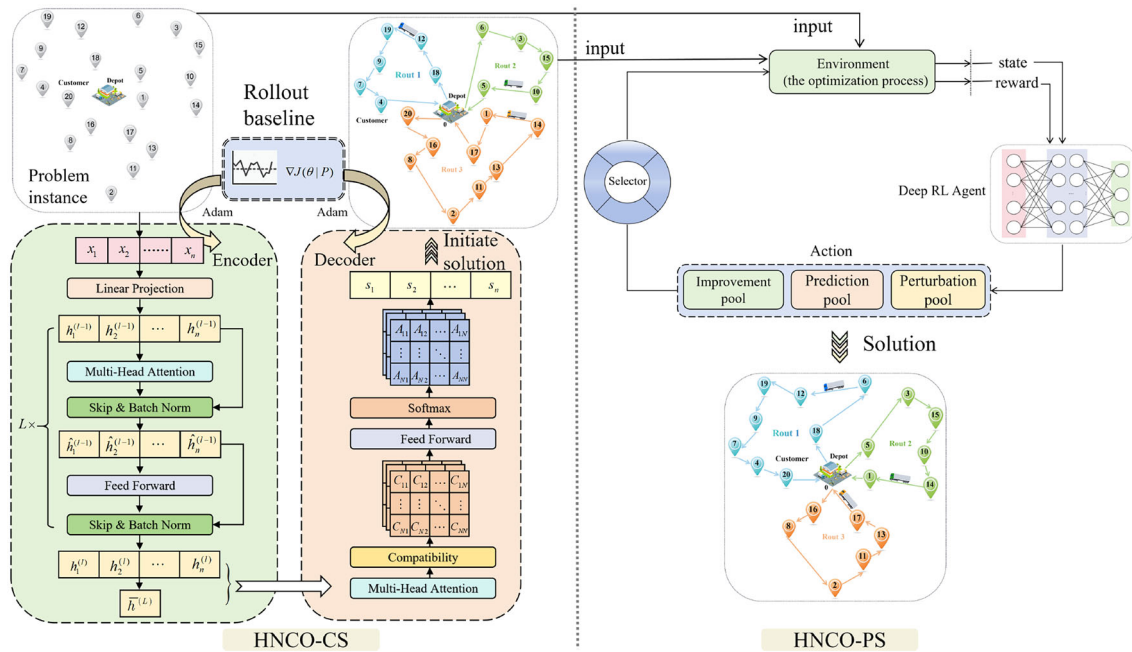


Fig. 1 HNCO Framework. (Left) Constructive Stage of the Framework, (Right) Perturbative Stage of the Framework

HNCO Framework

The proposed HNCO framework contains two stages, the HNCO-CS and the HNCO-PS, which are shown as the left and right part of Fig. 1, respectively. The HNCO-CS involves a Transformer-based encoder and decoder, the objective is to construct a reasonable and feasible initial solution based on prior knowledge. The HNCO-PS is driven by a DRL network that cooperates with three heuristic pools, namely, the improvement pool, prediction pool, and the perturbation pool. The goal of the HNCO-PS is to iteratively improve the solution constructed by the HNCO-CS using the idea of automated algorithm design. The details of each stage are as follows.

Constructive stage

The Constructive Stage utilizes the AM [1] to construct a suitable solution that will serve as the initial solution for the next stage. We adhere to the original heavy encoder and light decoder AM model without making any modifications. The AM consists of an encoder and a decoder, where the encoder has L attention layers and the decoder has one attention layer. Take CVRP as an example, the encoder takes the features of the problem instance as input, e.g., the coordinates of nodes in a CVRP instance, and outputs node embedding and graph embedding. The decoder takes the node embedding and graph embedding, along with the problem-specific mask and context, as input, and produces a sequence of nodes (a solution of the CVRP instance), denoted as $S = \{s_1, \dots, s_2\}$,

$s_t \in \{1, \dots, n\}$, $s_t \neq s_{t'}$ if $t \neq t'$, $t \in \{1, \dots, n\}$, where s_t represents the t -th element in the solution. (1) defines the random policy $A(s|p)$ that constructs a solution s for the instance p based on the AM. It is decomposed and parameterized by θ through the chain rule.

$$A_{\theta}(S|p) = \prod_{t=1}^n A_{\theta}(s_t|p, s_{1:t-1}) \quad (1)$$

Herein, we only use a sampling strategy to obtain the solution $S|p$ for instance p as shown in (2), in which the objective loss $J(\theta|S) = \mathbb{E}_{A_{\theta}(S|p)}[L(S)]$ is optimized using the REINFORCE gradient [14] with a baseline $b(S)$. In (2), $L(S)$ represents the current cost, and $b(S)$ denotes the solution cost constructed by the best model achieved during the training process.

$$\nabla J(\theta|p) = \mathbb{E}_{S \sim A_{\theta}(\bullet|p)}[(L(S|p) - b(p)) \nabla \log A_{\theta}(S|p)] \quad (2)$$

Encoder

The purpose of the encoder is to extract the embeddings $h_i^{(l)}$ for node i and graph embedding $\bar{h}^{(l)}$ by layer $l \in \{1, \dots, L\}$ from the input instance data $x = \{x_i\}$, $i \in \{1, \dots, n\}$. Take the CVRP as an example, x_i represents the coordinates and demands of node i . The encoder consists of L attention layers with each consisting of two sub-layers, where the first sub-layer is a multi-head attention (MHA) structure and the

second sub-layer is a fully connected feed-forward (FF) network. Skip connections and batch normalization are used between the sub-layers. The encoder outputs the node embeddings $h^{(L)}$ and graph embedding $\bar{h}^{(L)} : \bar{h}^{(L)} = \frac{1}{n} \sum_{i=1}^n h_i^{(L)}$.

Decoder

Specifically, the decoder iteratively predicts the next node to visit, i.e. s_t , based on the node embeddings, graph embeddings from the encoder output, and $s_{t'}$ at time $t' < t$. Based on (2), the compatibility C_{ij} between the query q_i of node i and the key k_j of node j can be computed as (3), where d_k represents the dimensionality of the attention head. Finally, softmax is applied to output the probability vector $A : A_i = A_\theta(s_t = i | p, s_{1:t-1}) = \text{softmax}(C_{ij})$.

$$C_{ij} = \begin{cases} \frac{q_i^T k_j}{\sqrt{d_k}} & j \neq s_{t'}, \forall t' < t, \\ -\infty & \text{otherwise.} \end{cases} \quad (3)$$

Perturbative stage

The perturbative stage receives the output solution from the constructive stage as the initial solution, then iteratively improves it via heuristics that are selected by the heuristic selector until the stop condition is satisfied. This stage comprises three heuristic pools, namely, the improvement pool, the prediction pool, and the perturbation pool. The heuristic selector is responsible for choosing the most suitable heuristics from these pools according to the search state. Fundamentally, the heuristic selector is a policy network that is trained by DRL. It ranks heuristics based on the current state, forming an implicit ranking of heuristic preselection, and selects the suitable heuristic to apply to the current solution. Importantly, the ranks are not predetermined or static, instead, they are dynamically determined based on the states. Therefore, the workflow of the perturbative stage is roughly the training process of RL.

DRL is a kind of machine learning techniques, it is often modeled as a Markov Decision Process, which can be denoted as $M = (S, A, \pi, r)$, where S , A , π , and r represents the state of the environment, the action that agent can take, the policy of the agent, and the reward that agent can receive. At each time step t , the DRL agent takes an action $a_t \in A$ according to the policy $\pi(a_t | s_t)$. After that, the environment will move to a new state s_{t+1} , and obtain corresponding rewards $r_t \in r$. The objective of DRL is to train a policy that can lead the agent achieve maximum accumulated reward. To be more general, we treat this stage as an automated algorithm design task that is in accord with the GSF definition in [8]. The designed states, actions, and rewards are detailed in follows.

States

Each state encompasses features of the problem instance, the current solution, and the historical records of heuristics. The problem instance features are fixed during the iterative process, which describe the inherent characteristics such as customer locations and demands in CVRPs. The solution vector represents the currently constructed solution, and the historical records of heuristics are the sequences of the selected heuristics and their corresponding rewards.

Actions

The three heuristic pools, namely, the improvement pool, prediction pool, and the perturbation pool, form the action space. Each heuristic pool contains different heuristics, and each heuristic has a distinct functionality. Details of the these heuristic pool are elaborated as follows.

The purpose of the improvement pool is to fine-tune the current solution using local search heuristics. Local search heuristics excel in optimizing local regions of the solution, such as the 2-Opt. By meticulously optimizing these local regions, the solution quality can be incrementally improved, gradually approaching the global optima. As shown in Table 1, the improvement pool consists of 8 types with a total of 27 heuristics, and indexes of heuristics range from 1 to 27.

The heuristics in the prediction pool are algorithms based on NNs, which aim to capture characteristic information of historical local optima during the iterative process and predict potential solutions. As shown in Table 2, there are 3 heuristics, namely, the RNN, GRU, and LSTM in the predication pool, whose indexes range from 28 to 30. These heuristics, by learning patterns and regularities from historical solution sequences, are capable of capturing the local structures and characteristics of the problem, particularly in capturing the local features and long-term dependencies among local optima. Based on the learned historical patterns and regularities, they can predict promising solutions, guiding the heuristics towards promising search directions.

As illustrated in the Fig. 2, the DRL agent autonomously selects appropriate prediction heuristics from the prediction pool. These heuristics leverage their memory capabilities to select a suitable number of local optima from the dataset of local optimal solutions for training. Subsequently, they infer based on the learned features of these local optima to predict a solution, guiding the next step of the search process. Conversely, when the predicted solution is inferior, its function is akin to that of the perturbation pool. By capturing the dependencies among sequences of local optima, these heuristic uncover the hidden node order relationships among the local optima, thereby supporting automated algorithm design.

Table 1 Improvement pool. The numbers within parentheses represent the number of paths affected by the heuristic

Heuristic	Name	Description
h_1	2-Opt (1)	Remove two edges and reconnect their endpoints
h_2	Exchange (1)	Exchange two customers in the route
h_5, h_6, h_7, h_{24}	Symmetric-exchange (2)	Exchange segments of length m ($m = 1, 2, 3, 4$) between two routes. Specifically, exchange the same number of customers between two paths, with the number of customers exchanged by the heuristic being 1, 2, 3, 4 in sequence
$h_{12 \sim 23}$	Asymmetric-exchange (2)	Exchange segments of length m and n ($m = 1, 2, 3, 4, n = 1, 2, 3, 4, m \neq n$) between two routes. Specifically, exchange unequal numbers of customers between two paths. The difference between heuristics lies in the number of customers exchanged, which varies from 1 to 4
$h_3, h_{25}, h_{26}, h_{27}$	Relocate (1)	Move m ($m = 1, 2, 3, 4$) customers in the route to new locations. Move the positions of m customers on a single path, with the number of customers moved being 1, 2, 3, 4 in sequence
h_8, h_9, h_{10}	Relocate (2)	Move a segment of length m ($m = 1, 2, 3$) from a route to another. The quantity of customers relocated by h_8 and h_9 is both 1, with the only distinction being that the source path and destination path are precisely reversed. The quantity of customers relocated by h_{10} is 2 or 3
h_4	Cross (2)	Exchange the tails of two routes
h_{11}	Cyclic-exchange (2)	Exchange cyclically one customer between two routes

Table 2 Prediction pool

Heuristic	Name	Description
h_{28}	RNN	Predict a solution based on the previous local optima. The difference lies in the memory capacity of the model, which means that the number of local optimal solutions that can be stored is different.
h_{29}	GRU	
h_{30}	LSTM	

Specifically, the employed RNN heuristic consists of 4 RNN units and 2 fully connected layers. The structure of RNN unit is illustrated in Fig. 3. As for the GRU heuristic, it comprises 3 GRU units and 3 fully connected layers. The structure of the GRU unit is depicted in Fig. 4. At last, the LSTM heuristic encompasses 4 LSTM units and 1 fully connected layer, and the structure of the LSTM unit is shown in Fig. 5. The specific parameters of these three heuristics are detailed in Table 2.

As for the perturbation pool, it contains heuristics that can change solutions dramatically to avoid being trapped in local optima. Table 3 shows three perturbative heuristics that used in this paper, whose indexes range from 31 to 33.

By intelligently integrating the three types of heuristic pools, we can leverage the strengths of traditional heuristics in local search and NN heuristics in feature learning and prediction, resulting in the following benefits:

- (1) **Complementary Advantages:** The heuristics in the improvement pool focus on local exploration and more like local search algorithms, heuristics in the perturbation pool excel at extensive perturbations and work like global search methods, while NN heuristics in the prediction pool excel at feature learning and solution prediction, and works flexibly as either local or global search. By intelligently choosing these heuristics, a comprehensive optimization strategy can be formed, i.e., it can flexibly and intelligently select appropriate heuristics to perform either local or global search to efficiently explore the search space.
- (2) **Multi-scale Search:** Different heuristics operate at different scale level as stated above, thus forming a multi-scale search structure, which may be more robust as tackling COPs with various scales.
- (3) **Adaptive Optimization:** It can achieve a balance between exploration and exploitation by integrating improvement, prediction, and perturbation heuristics. The agent adaptively adjusts the heuristic selection policy based on current state, automatically select the most suitable heuristic for optimization, thereby enhancing the effectiveness in handling various COPs.

During the process of model training, the automatic switching between heuristic pools is performed based on trial-and-error learning. In the iterative process, if the heuristics in a certain heuristic pool fail to further reduce the cost after a certain number of steps, it automatically switches to another pool. The default order of pool usage is the improvement pool, prediction pool, and perturbation pool, with switching thresholds set to 6, 3, and 3 consecutive steps, respectively. For instance, if no improvement in solution

Fig. 2 Proposed prediction pool for automated algorithm design. At each time step t , data sampled from the dataset forms the data pipeline, which has dimensions of (batchsize, sequencelength, features). Each slice of the data pipeline represents a batch of sequences of local optimal solutions, where the batch size denotes the number of batches in the input data, sequence length indicates the length of a local optimal solution, and features represent the number of features input at each time step. For the CVRP, each feature vector includes the coordinates of nodes, demand quantities, distance matrices between nodes, and features related to node indices

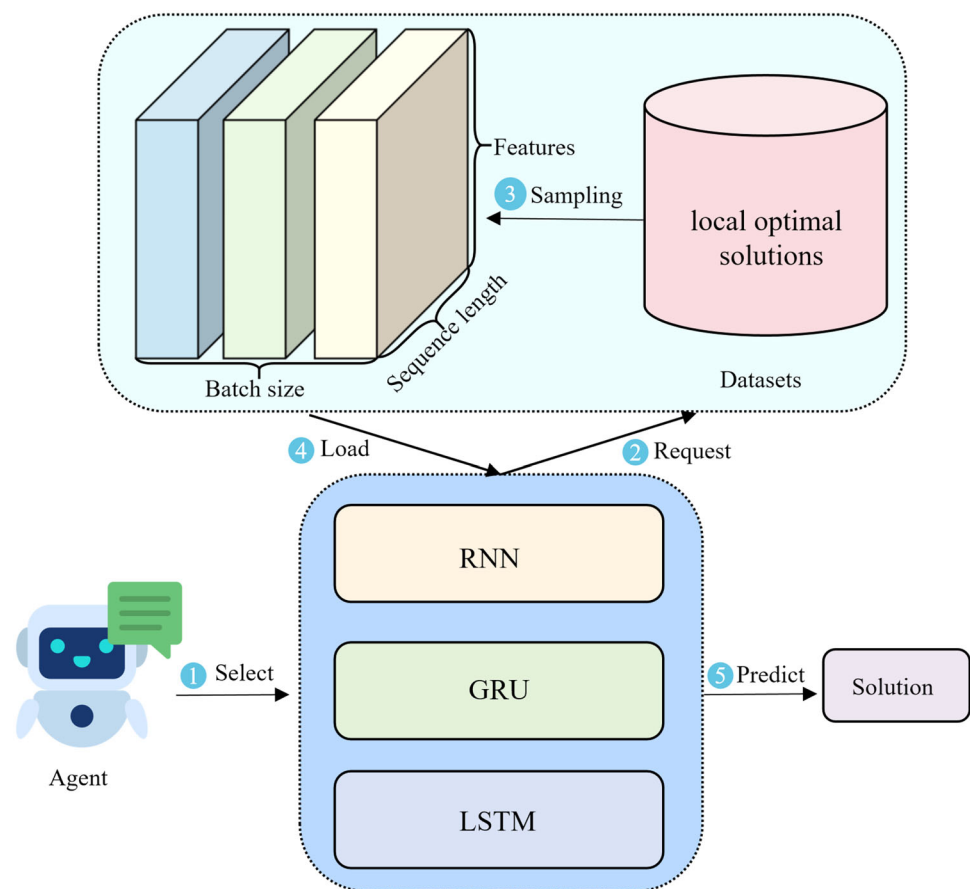
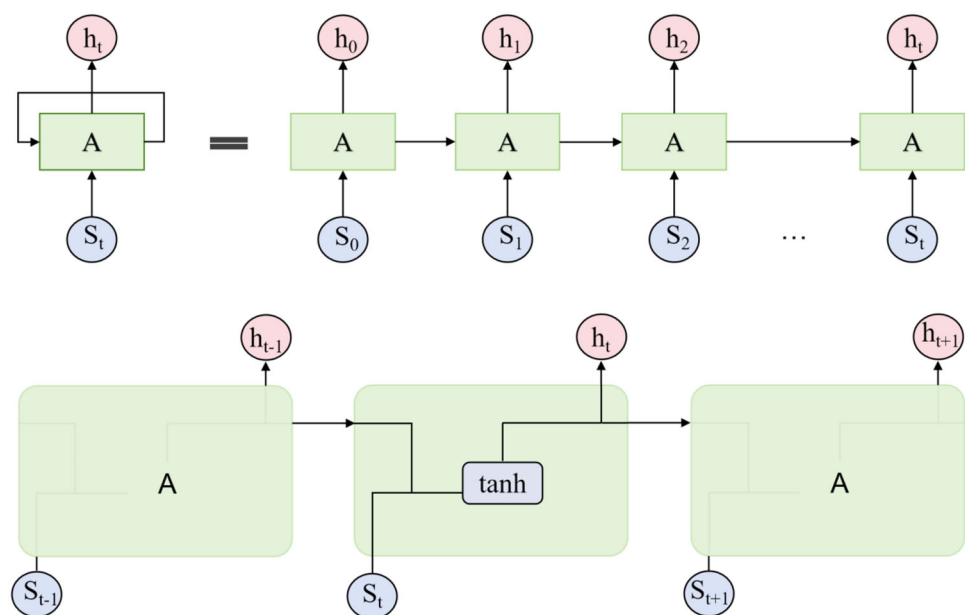


Fig. 3 The structure of the basic RNN unit. it includes one hidden layer. At each time step t , it recursively combines the input data S_t at the current time step with the hidden state h_{t-1} from the previous time step, passing this combination to the hidden state h_t at the current time step. This process captures and models the dynamic dependencies among local optima



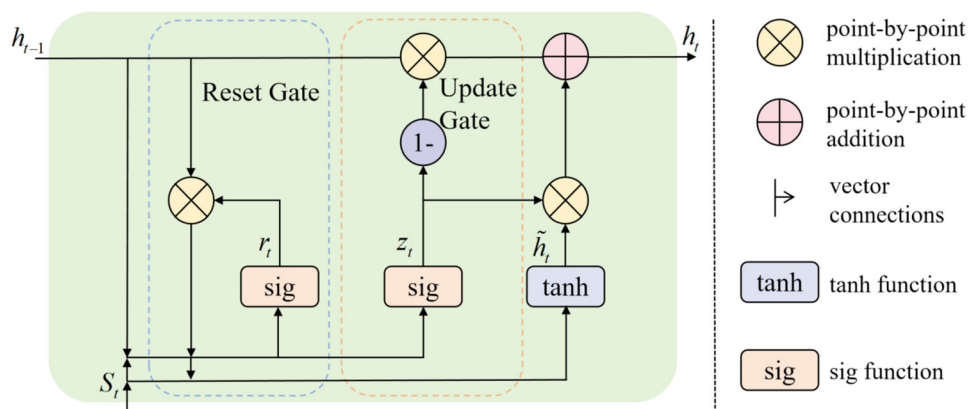


Fig. 4 The structure of the basic GRU unit. It involves four key components, i.e., the reset gate r_t , update gate z_t , candidate hidden state $\tilde{h}_t$, and the current hidden state h_t . At each time step t , the reset gate r_t controls the retention of past hidden state information h_{t-1} , while the update gate z_t regulates the flow of information between past and

current states. The candidate hidden state $\tilde{h}_t$ captures new information based on the current input S_t and past hidden state, and the final hidden state h_t is a combination of the past and candidate states determined by the update gate z_t

Fig. 5 The structure of the basic LSTM unit. It comprises three gate units: the input gate (i_t), forget gate (f_t), and output gate (o_t). At each time step t , S_t denotes the input vector, $\hat{C}_t$ represents the candidate cell state vector, C_t is the cell state vector, and h_t is the hidden state vector computed based on C_t . The return vectors corresponding to the three gate units are i_t , f_t , and o_t , respectively

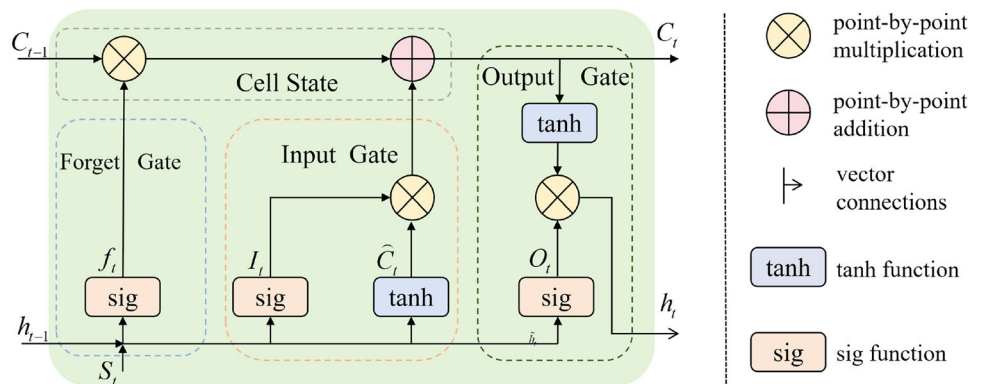


Table 3 Perturbation pool

Heuristic	Name	Description
h_{31}	Random-permute	Randomly destroy m ($m = 1, 2, 3$) routes and re-construct them by visiting affected customers in a random order
h_{32}	Random-exchange (2)	Randomly exchange m ($m = 1, 2, 3$) pairs of nearby customers between two routes
h_{33}	Cyclic-exchange	Exchange cyclically customers between multiple routes

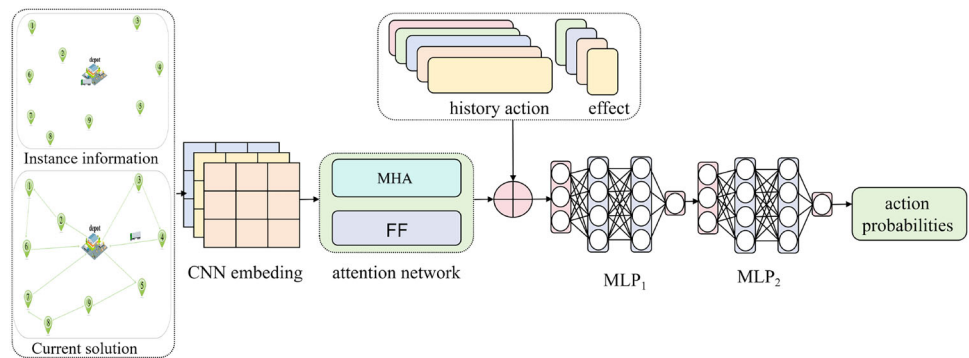
quality is observed after 6 consecutive steps by using the improvement pool, the usage of the improvement pool will be terminated and it will switch to the prediction pool, and etc.

Rewards

The reward consists of two components: the Quality Improvement Reward and the Baseline Reward. The Quality Improvement Reward incentivizes the agent to discover higher quality solutions by providing rewards proportional to the extent of the improvement. If a new solution is significantly distinct

and better than previous solutions, an additional reward is given to encourage exploration. The Baseline Reward compares the current solution to a predefined baseline solution. If the current solution surpasses this baseline, a reward is calculated using a weighting factor α to adjust the magnitude, thereby reinforcing the agent's progress toward optimal solutions.

1. **Quality Improvement Reward:** When the agent discovers a solution that is better than the previous one during the current iteration, a positive reward is given. The magnitude of the reward is proportional to the extent of the

Fig. 6 Adaptive Policy Network

improvement. For example, if the improved solution is 10% better than the previous one, a reward equal to 10% of the cost difference between the two solutions is granted. Additionally, to encourage the agent to explore novel solutions, if the new solution is distinct from all previous solutions and of higher quality than the previous one, the agent receives a reward of +1; otherwise, it receives a reward of -1.

2. **Baseline Reward:** A comparison is made between the current solution $C_{current}$ and the baseline solution $C_{baseline}$ (the initial solution from the HNCO-CS stage is used as the baseline solution). If the current solution surpasses the baseline solution, a reward r_t shown as (4) where α is the weighting factor used to adjust the magnitude of the rewards, is granted.

$$r_t = \alpha \cdot \frac{C_{baseline} - C_{current}}{C_{baseline}}, t \in \{h_1, h_2, \dots, h_{33}\} \quad (4)$$

Policy network

As shown in Fig. 6, the implicit ranking between heuristics is generated by the adaptive policy network. This NN consists of an attention network and two multilayer perceptron networks. It takes the current state as input and trains the network weights using policy gradients. Ultimately, the network outputs a probability vector that determines the selection of actions, enabling the automatic improvement of the current solution using problem-specific heuristics for different problem instances.

Experiments

In this section, we will introduce the experimental setup, including parameter settings, comparative experiments, and their analysis based on the TSP and CVRP datasets. To ensure the scientific validity of the experiments, we adopt the same parameter settings as previous studies [1, 29]. Specifically, in terms of the synthetic CVRPs, we set the customer numbers

Table 4 Model and training parameters of HNCO-CS

Model parameters	Training parameters
Encoder: 3 layers	Dataset: $20 \times 2500 \times 512$
Decoder: 1 layer	Decoding method: Sampling
FF: 512	Sample size: 1280
Optimizer: Adam [34]	Learning rate: $\eta_1 = 10^{-4}$

to 20, 50, and 100, and the corresponding vehicle capacities to 20, 30, and 40, respectively. The positions of customers and warehouses are uniformly sampled from a unit square, i.e., (x_i, y_i) , $x_i, y_i \in [0, 1]$, while each customer demand q_i is a random integer in $[1, 9]$. Transportation costs between two positions are calculated as Euclidean distance. Herein, unless otherwise specified, the experimental parameters are presented in Tables 4, 5, and 6.

In Table 4, we present the model parameters and training parameters for HNCO-CS, where the model parameters include the number of layers in the encoder and decoder, the dimensionality of the FF layer, and the used optimizer. The training parameters encompass the size of the dataset, the decoding method of the decoder, the number of samples during decoding, and the learning rate for DRL. In Tables 5 and 6, we respectively introduce the model parameters for the neural heuristics in the prediction pool and the training parameters for HNCO-PS. The model parameters include the network of neural heuristic (FC in Table 5 represents the fully connected layer), the number of neurons, dropout rate, activation function, loss function, and optimizer, along with the model parameters for the DRL policy network. In Table 6, training parameters for DRL, codes for heuristics, and the frequency of heuristic transformation in the training heuristic pool are provided.

To ensure the reliability of the results, 4000 iterations are conducted for each CVRP instance and the solution with the shortest tour is selected as the output. Note that the fairness of stopping criteria is indeed a significant and complex issue for PNCO, a higher number of iterations generally leads to a longer search process that may result in better outcomes. Given that the proposed HNCO integrates both

Table 5 Model parameters of HNCO-PS

Parameters	RNN	GRU	LSTM
Network	RNN(4)+FC(2)	GRU(3)+FC(3)	LSTM(4)+FC(1)
Neuron	64 128 256 512	64 128 256	64 128 256 512
Dropout rate	20% 30% 40%	20% 30% 40%	20% 30% 40% 40%
Activation function	Tanh (Network layer) + Relu (FC layer)		
Loss function	Mean squared error		
Optimizer	Adam [34]		
Reward adjustment factor	$\alpha = 0.5$		
Policy network	CNN (1) + Attention layer (1) + MLP (2)		

Table 6 Training parameters of HNCO-PS

Parameters	Values
Discount factor	1.0
Learning rate	$\eta_2 = 10^{-3}$
Epsilon-greedy	5.0×10^{-2}
Heuristic indexes	$h_{1 \sim 33}$
Heuristic pool switch steps	3, 3, 6
Iteration count	4000

CNCO and HNCO, to ensure the fairness of comparisons between HNCO and other algorithms, we set the parameters in HNCO-CS and HNCO-PS consistent with AM and L2I, respectively, expect that L2I uses 40,000 iterations, which are much more than the HNCO.

Unless otherwise stated, each experimental result is calculated as the average tour length based on 10k random samples. Besides, we also validate the designed algorithm on real CVRP instances from CVRPLib [38] and TSP instances from TSPLib [41]. We run our algorithm on each real-world problem instance 200 times and the best result from these runs is selected to calculate the gap with the Best-Known Solution (BKS). The method is implemented in Python and accelerated using a RTX 4080 GPU.

Overall comparisons with other algorithms

We compare the HNCO with several representative algorithms, including two classical COP solvers LKH3 [6] and Google OR Tools [7], two CNCOs, i.e., the AM [1] and POMO [18], and three PNCOs, namely, the NeuRewriter [35], NLNS [36], as well as the L2I [33]. The experimental results on the synthetic CVRP instances and the instances from TSPLib and CVRPLib are shown in Tables 7, and 8, respectively. In Table 7, the N, Obj., Gap, and Time represents the number of customs in a CVRP instance, the average tour length of obtained solutions, the gap calculated with the Obj. of LKH3 [6], and the time consumed by solving all 10K samples, respectively.

From Table 7 we can see that the designed HNCO achieved the minimum Obj. and Gap in all three groups of test sets among all compared algorithms. This verifies the effectiveness of the designed HNCO. However, its efficiency is inferior to other algorithms, namely, it consumes much more time than other algorithms. The key reason may be the utilization of the prediction pool, in which the heuristics need to be trained via supervised learning that is inevitably time-consuming.

To further verify the performance of HNCO, we compare it with two state-of-the-art learning-based algorithms: the POMO [18] and the LEHD [20] on CVRPLib test sets. Both the POMO and the LEHD are recently proposed outstanding CNCO framework, where the POMO has a heavy encoder and a light decoder while the LEHD is the opposite. Benefit from the Random Re-Construct (RRC) mechanism that tries to iteratively refine the solutions, the LEHD is superior in handling large-scale problems with more than 100 points while it also has excellent performance in small-scale problems as the POMO does [18]. In Table 8, A, B, E, F, M, P, and X represents different set of the instances, values in the bracket means the range of customer number, and the symbol # indicates the number of instances in corresponding instance sets. The LEHD with greedy and RRC-1000 means the versions that do not employ RRC and employ the RRC for 1000 iterations, respectively. The percent values represent the average gaps of all instances in the instance set.

As can be seen from Table 8, HNCO outperforms the majority of compared algorithms on the TSPLib instances and achieves the smallest average gap across all CVRPLib instances. This clearly demonstrates its excellent generalization capability and robust performance in handling COPs. Interestingly, from Table 8 we can also see that the LEHD with greedy strategy has relatively poor performance compared with that with the RRC strategy. This indicates that it is the RRC which performs like a heuristic that actually endows LEHD with the potential for sustaining optimization capability. Therefore, LEHD with RRC strategy is more like a two-stage method instead of a pure NCO as [20] claimed. Even though, the proposed HNCO still outperforms

Table 7 Comparison of Our Experimental Results with Prior Works

Method	Obj Gap		CVRP N=50			CVRP N=100			Time
			Time	Obj	Gap	Time	Obj	Gap	
LKH3	6.12	–	2 h	10.38	–	7 h	15.68	–	12 h
OR Tools	6.42	4.48%	2 m	11.22	8.12%	12 m	17.14	9.34%	1 h
NeuRewr- iter	6.16		22 m	10.51		18 m	16.10		1 h
NLNS	6.19		7 m	10.54		24 m	15.99		1 h
AM greedy	6.40	4.45%	<1 s	10.93	5.34%	1 s	16.73	6.72%	3 s
AM sampling	6.24	1.97%	3 m	10.59	2.11%	7 m	16.16	3.09%	30 m
POMO, trajec	6.35	3.72%	<1 s	10.74	3.52%	1 s	16.15	3.00%	3 s
POMO, no aug	6.17	0.82%	1 s	10.49	1.14%	4 s	15.83	0.98%	19 s
POMO, ×8 aug	6.14	0.21%	5 s	10.42	0.45%	26 s	15.73	0.32%	2 m
L2I	6.12	0.00%	45 h	10.35	−0.29%	82 h	15.57	−0.70%	236 h
HNCO	5.76	−5.88%	82 h	9.65	−7.03%	108 h	14.69	−6.31%	386 h

Table 8 Experimental Results on TSPLib and CVRPLib

	Set(size)	#	POMO aug×8	LEHD greedy	RRC-100	HNCO sampling
TSPLib	<100	6	0.792%	0.976%	0.481%	0.269%
	100-200	21	2.423%	2.336%	0.158%	0.5296%
	200-500	15	13.413%	2.742%	0.200%	0.274%
	500-1k	6	31.678%	4.049%	1.310%	1.295%
	>1k	22	63.705%	11.267%	4.088%	0.403%
	All	70	26.406%	5.260%	1.529%	0.554%
CVRPLib	A(31-79)	27	4.970%	5.871%	0.647%	0.411%
	B(30-77)	23	4.747%	6.049%	0.812%	0.465%
	E(12-100)	11	11.402%	4.809%	0.541%	0.088%
	F(44-134)	3	15.973%	9.051%	3.009%	1.569%
	M(100-199)	5	4.861%	7.094%	1.817%	0.219%
	P(15-100)	23	15.525%	6.611%	0.917%	0.703%
	X(100-1k)	100	21.684%	12.520%	3.511%	3.488%
	All	192	15.450%	9.465%	2.253%	0.991%

LEHD with 1000 RCC iterations, which is the best reported NCO framework until now as we know. In conclusion, these results demonstrate the good performance and robustness of HNCO. However, it is noteworthy that in our experiments, the HNCO-PS stage undergoes real-time training during the search process, which may contribute to the high computational cost.

The effectiveness of the hybrid

To verify the necessity of each stage, we compare the performance of HNCO with its two versions, namely, HNCO with merely the constructive stage (HNCO-CS), HNCO with merely the perturbative stage (HNCO-PS), and the randomly generated solutions (RGS). Although both HNCO-CS and RGS are utilized to construct initial solutions for HNCO-PS,

the difference lies in that the former utilizes the AM while the latter directly generates initial solutions in a random manner. The parameters are consistent with the above settings. Table 9 shows the comparison results on CVRP 20, 50, and 100 test sets. According to the results, HNCO reaches the best performance in all test sets, demonstrating the superiority of the hybrid framework. Moreover, comparisons between HNCO-CS and HNCO-PS shows that the perturbative stage is more powerful than the constructive stage. However, HNCO-PS is still inferior to HNCO, indicating that the constructive stage is still indispensable to the HNCO framework.

The effectiveness of the perturbation pool

To investigate the impact of perturbation pool on the HNCO framework, we consider scenarios without using a perturba-

Table 9 Comparisons among different HNCO versions

	CVRP N=20	CVRP N=50	CVRP N=100
RGIS	10.62	23.45	43.54
HNCO-CS	6.44	10.93	17.69
HNCO-PS	5.84	9.95	14.97
HNCO	5.76	9.65	14.69

tion pool (w/o), with random selection heuristics (Random), and with rule-based prediction pool (Rule). Here, w/o signifies the absence of perturbation pool usage, where the heuristic pool only encompasses the improvement pool and prediction pool. The term Random denotes that the model will randomly select heuristics from the perturbation pool, while the term Rule refers to select heuristics based on the ratio between the current solution and the historical local optimum. Specifically, the ratio R is calculated as (5), where $C_{current}$ and C_{best} represent the cost of the current solution and the local optimum, respectively. Then based on the value of R , heuristics h_{32} , h_{33} , and h_{31} from the perturbation pool are selected by following rules.

$$R = \frac{C_{current}}{C_{best}} \quad (5)$$

- When $R \in (0, 0.5]$, it implies that the current solution has already surpassed the local optimum by a large margin. Therefore, in this scenario, we opt for h_{32} to introduce minor perturbations to the current solution.
- When $R \in (0.5, 1]$, it implies that the current solution is also better than the local optimum but by a smaller margin compared with the above scenario. Therefore, we choose h_{33} with a moderate disturbance amplitude to introduce moderate perturbations to the current solution.
- When $R \in (1, +\infty)$, it indicates that the current solution is relatively worse than the local optimum, suggesting that the current solution deviates from the local optimum solution with room for improvement. Therefore, We employ h_{31} with a large perturbation amplitude to introduce significant perturbations to the current solution, aiming for notable improvements.

In summary, the Rule utilizes the ratio between the current solution and the historical local optimum to determine which heuristic to use, ensuring that the perturbation amplitude is appropriate for the current solution and avoiding premature convergence. The comparison results are presented in Table 10, from which it can be seen that the w/o version is overall inferior to the other three versions, which demonstrates that the usage of the perturbation pool is beneficial to HNCO. Furthermore, by comparing the RL version with the

Table 10 Selecting heuristics from the perturbation pool using different strategies

	CVRP N=20	CVRP N=50	CVRP N=100
w/o	6.00	10.97	16.74
Random	6.08	10.41	15.73
Rule	6.08	10.42	15.50
RL	5.76	9.65	14.69

Random and Rule versions, it is clearly that the RL-based strategy is more suitable for heuristic selection.

The effectiveness of the prediction heuristics

To investigate the contribution of the prediction heuristics to the HNCO framework, we compare HNCO with its another version that excludes the prediction heuristics (denoted as HNCO-WP) on CVRP instances of size 20, 50, and 100. The comparison results are shown as Fig. 7, where the Cost is over 2000 instances for each size of CVRP instance, each epoch represents one problem instance. From Fig. 7 we can see that compared with HNCO-WP, HNCO can achieve overall lower cost on all three types of instances, which demonstrates that the utilization of the prediction heuristics can lead to better performance. However, note that the cost of solutions of HNCO exhibits significant fluctuations during optimization, which could be attributed to the instability of the heuristics in prediction pool.

In addition, to analyze the impact of the prediction heuristics on HNCO's time efficiency, we compare the time cost by HNCO (marked with * in Fig. 8) and HNCO-WP on CVPRs with the size of 20, 50, and 100. Figure 8 shows the averaged time over 500 random CVRP instances, where the time cost by each stage, i.e., the HNCO-CS and the HNCO-PS, is given separately to better understand their behavior. From Fig. 8 we can see that with the increase of problem size, the time consumed by both HNCO and HNCO-WP increase exponentially. This increase is mainly caused by the HNCO-PS. Meanwhile, HNCO consumes more time than HNCO-WP, which indicates that the prediction heuristics incurs additional time costs. The reason is that compared to traditional perturbation heuristics (e.g., 2-OPT), NN-based heuristics (e.g., RNN) require extensive parameter training and inference processes, and the training data for the heuristics is collected in real time. In summary, the introduction of prediction heuristics into the perturbation pool can improve the performance of HNCO but with extra time load.

Analysis of the learned algorithm components

To comprehensively understand what the HNCO can learn, we track the usage of each heuristic during the optimiza-

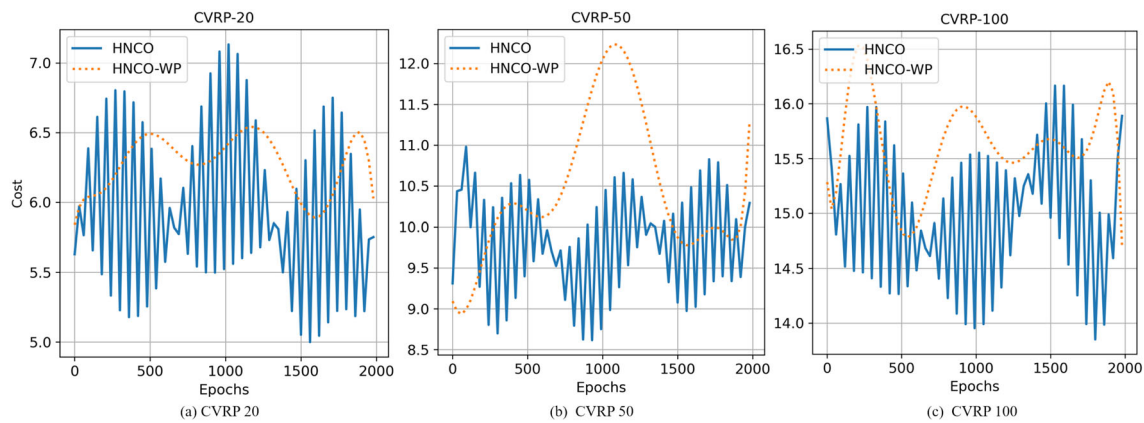


Fig. 7 Ablation experiment on the prediction pool

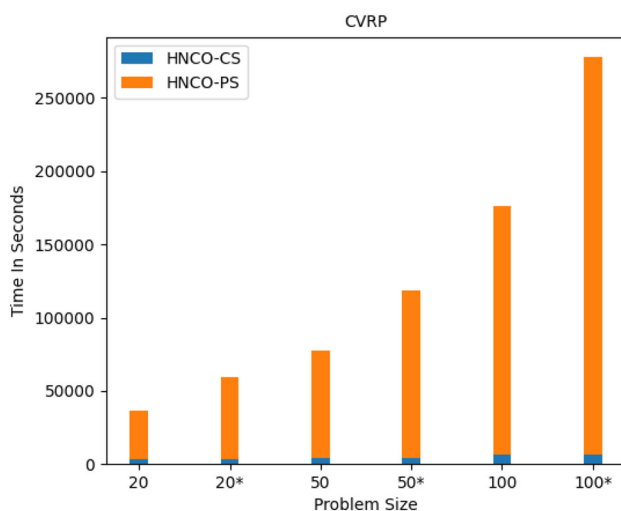


Fig. 8 The average time cost under different problem size

tion of CVRP with 20 customers. The averaged percent of each heuristic over 10k instances with each instance undergoing 40,000 iterations are shown as Fig. 9. From Fig. 9 we can clearly see that the improvement pool is used the most frequently. Specifically, in the perturbative stage, nearly 85% of the time is calling the heuristics in the improvement pool, in which heuristics 2, 7, and 13, namely, Exchange (1), Symmetric-exchange (2), and Asymmetric-exchange (2) are the most frequently called heuristics. This is reasonable because heuristics in the improvement pool are essentially designed to undertake the task of improving the current solution. Additionally, it is consistent with findings in other literature [29] that the Exchange method, Relocate method, and 2-Opt are identified as the most frequently called heuristics. Moreover, it should be noted that heuristics with low frequency of usage does not imply a lack of positive contribution to the search of better solutions.

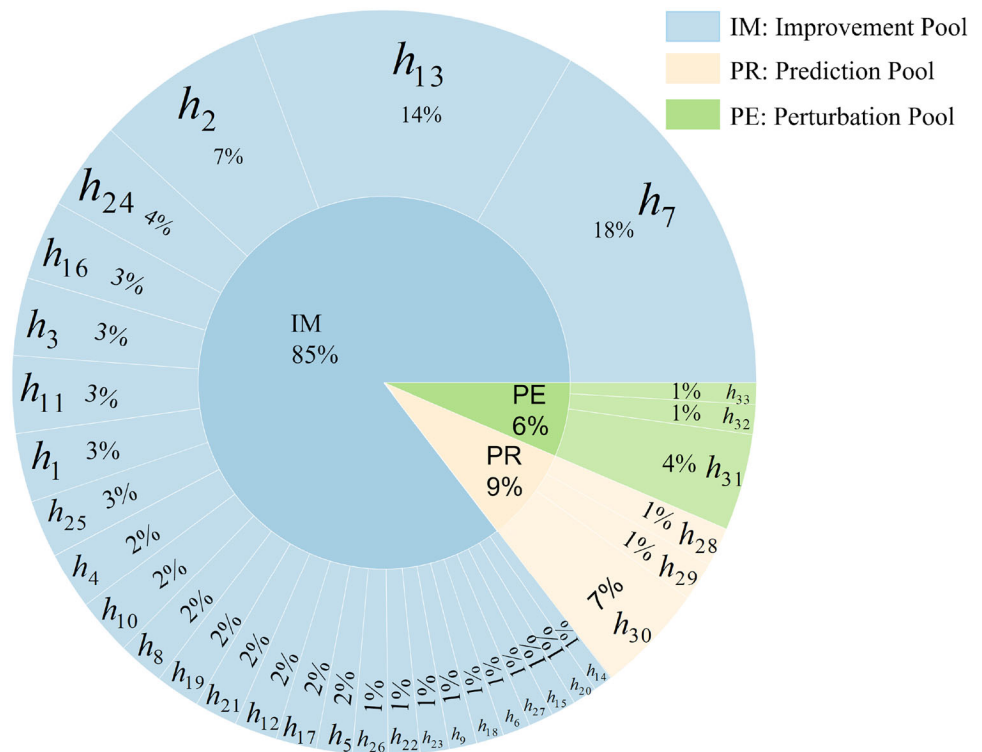
As for the other two heuristic pool, similar phenomena can be observed from Fig. 9. LSTM and Random-permute are the

most frequently called heuristics in the prediction pool and perturbation pool, respectively. However, it does not imply that GRU and RNN in prediction pool and Random-exchange (2) and Cyclic-exchange in perturbation pool contribute nothing for the optimization. On the contrary, the search space of COPs always comprise rugged search spaces [37] and different heuristics exhibit diverse search behaviors, namely, no heuristic can work perfectly on all COPs. Consequently, a search algorithm solely relying on a single heuristic may encounter low efficiency in solving COPs. To enhance performance, it is imperative to judiciously combine different algorithmic components to automatically design the search algorithms as HNCO does.

Computational complexity analysis

Due to the pre-training involved in the HNCO-CS and the online training in the HNCO-PS, along with the extensive set of heuristics in the heuristic pool, theoretically analyzing the time complexity is quite complex. Therefore, we made statistics on the time and proportions spent on each part of the process throughout the experiments for training and reference. Specifically, HNCO-CS undergoes pre-training on 25,600,000 CVRP problem instances, which costs approximately 13, 30, and 72 h for instances with 20, 50, and 100 nodes, respectively. The time spent during the entire optimization stage is detailed in Table 7. The HNCO-CS accounts for 6.33% ($N = 20$), 3.92% ($N = 50$), and 2.30% ($N = 100$) of the total runtime of the optimization stage. In contrast, HNCO-PS comprises 93.67% ($N = 20$), 96.08% ($N = 50$), and 97.70% ($N = 100$) of them. Within HNCO-PS, the runtime proportions of the improvement pool, prediction pool, and perturbation pool occupy 19.72%, 79.28%, and 1.00% of its runtime, respectively. Furthermore, within the prediction pool, RNN, GRU, and LSTM occupy 12.66%, 25.54%, and 41.08% of its runtime. From above we can know that its the HNCO-PS that is time consuming,

Fig. 9 The frequency of heuristic utilization



and within HNCO-PS, the prediction pool occupy the largest proportion of its runtime.

Conclusions

This paper proposes a hybrid neural combinatorial optimization framework, HNCO, that integrates the CNCO and PNCO methods into a whole. It consists of two stages, where the CNCO is the first stage and the PNCO is the second. In the first stage, we use the AM to quickly generate initial solutions for given COPs. These solutions can be obtained rapidly and have a near-optima quality. Then in the second stage, we utilize a NN trained with DRL to intelligently select the most appropriate heuristic from three heuristic pools, i.e., the improvement pool, prediction pool, and perturbation pool, to continuously optimize the obtained solutions. By leveraging the strengths of CNCO and PNCO, namely, high speed in constructing near-optima solutions and outstanding optimization capability, respectively, HNCO exhibits competitive performance both on synthetic CVRP instances and real CVRP instances from CVRPLib and TSP instances from TSPLib. Future research will further refine and extend the framework presented in this paper to address more challenging COPs such as timetabling and scheduling problems.

Acknowledgements This work is supported by the National Natural Science Foundation of China under Grant 62106199, the Key Research and Development Program of Shaanxi Province (No. 2020KW-068), and the General Project of Education Department of Shaanxi Provincial Government under Grant 22JK058.

Data availability The generated data in this study are available from the corresponding author on reasonable request.

Declaration

Conflict of interest The authors declare that they have no Conflict of interest.

Open Access This article is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License, which permits any non-commercial use, sharing, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if you modified the licensed material. You do not have permission under this licence to share adapted material derived from this article or parts of it. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by-nc-nd/4.0/>.

References

- Kool W, van Hoof H, Welling M (2019) Attention, learn to solve routing problems! In: 7th International conference on learning representations (ICLR). arXiv preprint <http://arxiv.org/abs/1803.08475>
- Veres M, Moussa M (2019) Deep learning for intelligent transportation systems: A survey of emerging trends. *IEEE Trans Intell Trans Syst* 21(8):3152–3168
- Du Y, Fu T, Sun J, et al (2022) Molgensurvey: a systematic survey in machine learning models for molecule design. arXiv preprint <http://arxiv.org/abs/2203.14500>
- Lloyd H, Amos M (2019) Solving Sudoku with ant colony optimization. *IEEE Trans Games* 12(3):302–311
- Li S, Luo T, Wang L et al (2022) Tourism route optimization based on improved knowledge ant colony algorithm. *Complex Intell Syst* 8(5):3973–3988
- Helsgaun Keld (2017) An extension of the lin-kernighan-helsgaun tsp solver for constrained traveling salesman and vehicle routing problems. Roskilde University, Roskilde
- Laurent Perron, Vincent Furnon (2023) Or-tools, URL <https://developers.google.com/optimization/>
- Yi W, Qu R, Jiao L et al (2023) Automated design of metaheuristics using reinforcement learning within a novel general search framework. *IEEE Trans Evol Comput* 27(4):1072–1084
- Yi W, Qu R (2023) Automated design of search algorithms based on reinforcement learning. *Inform Sci* 649:119639
- Zhong J, Feng Y, Tang S et al (2023) A collaborative neurodynamic optimization algorithm to traveling salesman problem. *Complex Intell Syst* 9(2):1809–1821
- Vinyals O, Fortunato M, Jaitly N (2015) Pointer networks. In: 2015 Neural information processing systems (NIPS), pp 2692–2700
- Zheng J, Wang L, Wang S et al (2021) Solving two-stage stochastic route-planning problem in milliseconds via end-to-end deep learning. *Complex Intell Syst* 7(3):1207–1222
- Bello I, Pham H, Le QV, Norouzi M, Bengio S (2017) Neural combinatorial optimization with reinforcement learning. In: 2017 International conference on learning representations (ICLR). arXiv preprint <http://arxiv.org/abs/1611.09940>
- Williams RJ (1992) Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Mach Learn* 8:229–256
- Gurobi Optimization, LLC (2023) Gurobi Optimizer Reference Manual. <https://www.gurobi.com>
- David A, Ribert B, Vasek C et al (2006) Concorde tsp solver. <http://www.math.uwaterloo.ca/tsp/concorde/>
- Deudon M, Cournut P, Lacoste A et al (2018) Learning heuristics for the tsp by policy gradient. 15th International Conference on Integration of Constraint Programming, Artificial Intelligence, and Operations Research (CPAIOR). Springer, Netherlands, pp 170–181
- Kwon YD, Choo J, Kim B et al (2020) Pomo: Policy optimization with multiple optima for reinforcement learning. *Adv Neural Inform Process Syst* 33:21188–21198
- Cheng H, Zheng H, Cong Y et al (2023) Select and Optimize: Learning to solve large-scale TSP instances. In: 26th International Conference on Artificial Intelligence and Statistics 206:1219–1231
- Luo F, Lin X, Liu F et al (2023) Neural combinatorial optimization with heavy decoder: Toward large scale generalization. arXiv preprint <http://arxiv.org/abs/2310.07985>
- Kim M, Park J (2021) Learning collaborative policies to solve NP-hard routing problems. *Adv Neural Inform Process Syst* 34:10418–10430
- Kwon YD, Choo J, Yoon I et al (2021) Matrix encoding networks for neural combinatorial optimization. *Adv Neural Inform Process Syst* 34:5138–5149
- Xin L, Song W, Cao Z et al (2020) Step-wise deep learning models for solving routing problems. *IEEE Trans Ind Inform* 17(7):4861–4871
- Xin L, Song W, Cao Z et al (2021) Multi-decoder attention model with embedding glimpse for solving vehicle routing problems. *Proc AAAI Conf Artificial Intell* 35(13):12042–12049
- Chen X, Tian Y (2019) Learning to perform local rewriting for combinatorial optimization. 33rd Conference on Neural Information Processing Systems (NIPS). Vancouver, Canada, pp 6278–6289
- Gao L, Chen M, Chen Q et al (2020) Learn to design the heuristics for vehicle routing problem. arXiv preprint <http://arxiv.org/abs/2002.08539>
- Schulman J, Wolski F, Dhariwal P et al (2017). Proximal policy optimization algorithms. arXiv preprint <http://arxiv.org/abs/1707.06347>
- Meng W, Qu R (2023) Sequential Rule Mining for Automated Design of Meta-heuristics. In: Proceedings of the Companion Conference on Genetic and Evolutionary Computation (GECCO). arXiv preprint <https://doi.org/10.1145/3583133.3596303>
- Meng W, Qu R (2021) Automated design of search algorithms: Learning on algorithmic components. *Expert Syst Appl* 185:115493
- Qu R, Kendall G, Pillay N (2020) The General Combinatorial Optimization Problem: Towards Automated Algorithm Design. *IEEE Comput Intell Mag* 15(2):14–23
- Yi W, Qu R, Jiao L (2023) Automated algorithm design using proximal policy optimisation with identified features. *Expert Syst Appl* 216:119461
- Hong L, Woodward JR, Özcan E et al (2021) Hyper-heuristic approach: automatically designing adaptive mutation operators for evolutionary programming. *Complex Intell Syst* 7(6):3135–3163
- Lu H, Zhang X, Yang S (2019) A learning-based iterative method for solving vehicle routing problems. In: 8th International conference on learning representations (ICLR)
- Kingma D P, Ba J et al (2014) Adam: A method for stochastic optimization. In: 3rd International conference on learning representations (ICLR) arXiv preprint <http://arxiv.org/abs/1412.6980>
- Chen X, Tian Y et al (2019) Learning to perform local rewriting for combinatorial optimization. In: 2019 Neural information processing systems (NIPS). arXiv preprint <http://arxiv.org/abs/1810.00337>
- Hottung A, Tierney K (2019) Neural large neighborhood search for the capacitated vehicle routing problem. In: 24th European Conference on Artificial Intelligence (ECAI). arXiv preprint <http://arxiv.org/abs/1911.09539>
- Meng W, Qu R (2021) Automated design of search algorithms: learning on algorithmic components. *Expert Syst Appl* 185:115493
- Uchoa E, Pecin D, Pessoa A et al (2017) New benchmark instances for the capacitated vehicle routing problem. *Euro J Oper Res* 257(3):845–858
- Zhou C, Lin X, Wang Z, et al (2024) Instance-Conditioned Adaptation for Large-scale Generalization of Neural Combinatorial Optimization. arXiv preprint [arXiv:2405.01906](https://arxiv.org/abs/2405.01906)
- Liu F, Tong X, Yuan M, et al (2024) Evolution of Heuristics: Towards Efficient Automatic Algorithm Design Using Large Language Model. arXiv preprint [arXiv:2401.02051](https://arxiv.org/abs/2401.02051)
- Reinelt Gerhard (1991) TSPLIB-a traveling salesman problem library. *ORSA J Comput* 3(4):376–384

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.